

Frontend Application Documentation

Intelligent Climate Control and Energy Management System with Olimex MOD-IO

Angular / TypeScript / PrimeNG

Editor: *Denislav Ivanov*

Table of Contents

1. Scope and Purpose 3

2. Technology Stack 3

3. Application Architecture 3

4. Implemented Functionalities 4

 4.1 Authentication and Session Management 4

 4.2 Main Dashboard 4

 4.3. Manual Control 4

 4.4. Threshold Configuration 4

 4.5 Notifications and Visual States 4

 4.6. Historical Charts 4

5. State Management and Data Flow..... 5

6. User Interface and Responsiveness 5

7. Startup and Deployment..... 5

1. Scope and Purpose

This document describes the frontend part of the system, including its architecture, main functionalities and startup process. The application is implemented as an Angular standalone application with a PrimeNG user interface. It provides user authentication, visualization of the current system status, mode control, manual fan control, temperature threshold configuration, and a chart for temperature and power supply monitoring.

2. Technology Stack

Technology	Role in the Project	Notes
Angular	Main frontend framework	Standalone components and router are used
TypeScript	Business logic	Services, models
PrimeNG	UI components	Card, Tag, Button, InputNumber, Toast, Chart
RxJS	Reactive data flow	Polling, catchError, finalize
Chart.js via PrimeNG Chart	Historical charts	Two axes: temperature and power supply

3. Application Architecture

The frontend application follows a separation of responsibilities:

- features/auth – login screen and authentication-related logic.
- features/dashboard – main dashboard with visualization, charts, and control actions.
- core/services – services for backend API communication and token management.
- core/guards – route protection for the dashboard.
- core/interceptors – automatic attachment of the Bearer token to protected requests.
- shared/models – TypeScript types for backend and database responses.
- environments – configuration for backend and DB API base URLs.

4. Implemented Functionalities

4.1 Authentication and Session Management

- A login form with username and password is used.
- After successful login, the access token is stored in localStorage through TokenService.
- The HTTP interceptor adds Authorization: Bearer <token> to all protected requests, except /login.

4.2 Main Dashboard

The dashboard displays:

- The current system mode: AUTO or MANUAL.
- The current temperature.
- The Power Supply value, formatted as volts.
- The time of the last successful update.
- The state of the three fans.

4.3. Manual Control

- Manual fan control is allowed only in MANUAL mode.
- The user can switch between AUTO and MANUAL mode.

4.4. Threshold Configuration

- The form supports three separate thresholds: Fan 1 Threshold, Fan 2 Threshold, and Fan 3 Threshold.
- PrimeNG InputNumber is used with minimum and maximum values and decimal formatting.
- When saved, the values are sent to the backend as query parameters and the backend sorts them in ascending order.

4.5 Notifications and Visual States

Global toast messages are displayed using PrimeNG p-toast.

- Notifications are shown for successful threshold saving, mode changes, and fan state updates.
- A warning notification is displayed when a high temperature is reached.
- The Power Alert is displayed as an overlay banner above the interface

4.6. Historical Charts

The chart combines two datasets: temperature and power supply.

- Two axes are used: the left axis for temperature and the right axis for power supply.
- On startup, historical data is loaded from the database using /api/history and /api/health.
- After that, new live points are added through polling to /status.
- If the database is empty, the interface continues working and starts accumulating points from the live data.

5. State Management and Data Flow

DashboardComponent uses Angular signals for status, loading, errorMessage, and lastUpdated.

- A reactive form is used for the threshold configuration.
- On initialization, loadInitialHistory() retrieves database data using forkJoin().
- After that, startLiveStatusPolling() calls /status every 2 seconds and updates the screen.
- The handleAlerts() method compares the previous and current state to avoid repeated unnecessary alert messages. The parsePowerSupply() method allows the frontend to work with both numeric values and text formats containing voltage information.

6. User Interface and Responsiveness

- The interface is implemented with PrimeNG and custom CSS.
- Fixed overlay alert banner positioned in the upper-right part of the screen.
- Full-width chart card for improved readability.

7. Startup and Deployment

- For production build, ng build --configuration production is used.
- The generated build is copied into backend/frontend, so FastAPI can serve the Angular application together with the API endpoints.
- Commands: ng build --configuration production, uvicorn main:app --reload